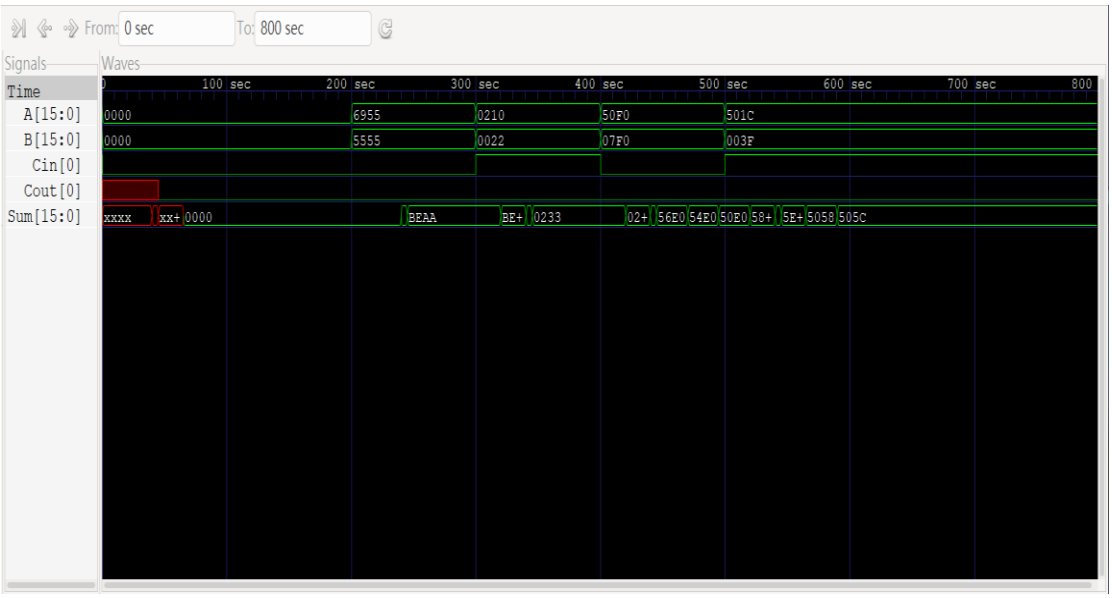
16-bit ripple carry adder (RCA)

```
module half_adder(output S, C, input x, y);
	xor #(20)(S, x, y);
	and #(10)(C, x, y);
endmodule

module full_adder(output sum, carry, input x, y, z);
	wire S1, C1, C2;
	half_adder ha1(S1, C1, x, y);
	half_adder ha2(sum, C2, S1, z);
	or #(15) G1(carry, C2, C1);
endmodule

module ripple_carry_4_bit_adder(output [3:0]s, output cout,	input[3:0] X, Y, input
Z);
	wire C1, C2, C3;
	full_adder fa1(s[0], C1, X[0], Y[0], Z);
	full_adder fa2(s[1], C2, X[1], Y[1], C1);
	full_adder fa3(s[2], C3, X[2], Y[2], C2);
	full_adder fa4(s[3], cout, X[3], Y[3], C3);
endmodule

module ripple_carry_16_bit_adder(output[15:0]aaa, output cout, input[15:0] A, B,
input cin);
	wire C1, C2, C3;
	ripple_carry_4_bit_adder rca1(aaa[3:0], C1, A[3:0], B[3:0], cin);
	ripple_carry_4_bit_adder rca2(aaa[7:4], C2, A[7:4], B[7:4], C1);
	ripple_carry_4_bit_adder rca3(aaa[11:8], C3, A[11:8], B[11:8], C2);
	ripple_carry_4_bit_adder rca4(aaa[15:12], cout, A[15:12], B[15:12], C3);
endmodule

module testbench_ripple_carry_16_bit_adder;
reg[15:0]A, B;
reg Cin;
wire[15:0]Sum;
wire Cout;
ripple_carry_16_bit_adder rca16bit(Sum, Cout, A, B, Cin);
```

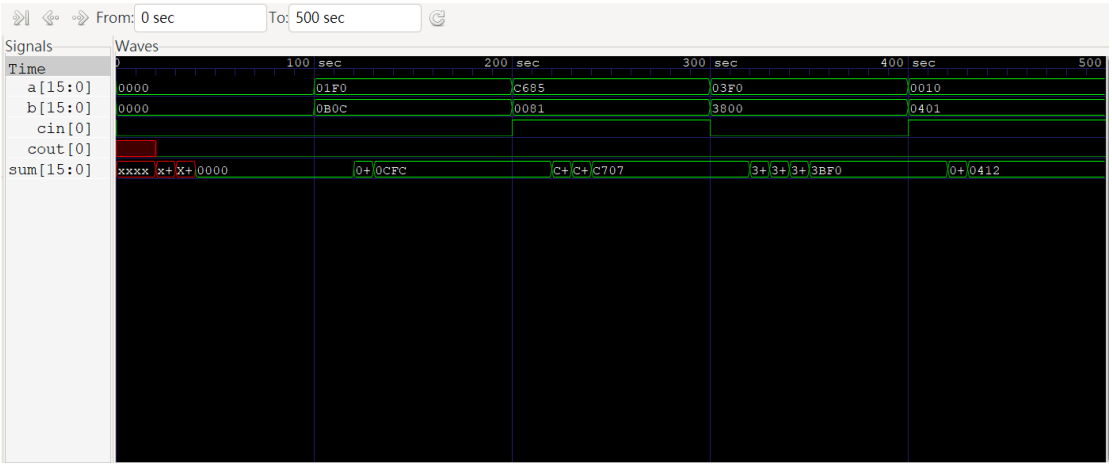
```
initial begin
A = 0; B = 0; Cin = 0;
#200 A = 16'b0110100101010101; B = 16'b0101010101010101; Cin = 1'b0;
#100 A = 16'b0000001000010000; B = 16'b0000000000100010; Cin = 1'b1;
#100 A = 16'b0101000011110000; B = 16'b0000011111110000; Cin = 1'b0;
#100 A = 16'b0101000000011100; B = 16'b0000000000111111; Cin = 1'b1;
end
initial #800 $finish;
initial $dumpvars;
endmodule
```



16-bit carry lookahead adder (CLA)

```
module carry_look_ahead_16_bit(input[15:0] a, b, input cin, output [15:0] sum,
output cout);
	wire c1,c2,c3;
	carry_look_ahead_4_bit cla1(a[3:0], b[3:0], cin, sum[3:0], c1);
	carry_look_ahead_4_bit cla2(a[7:4], b[7:4], c1, sum[7:4], c2);
	carry_look_ahead_4_bit cla3(a[11:8], b[11:8], c2, sum[11:8], c3);
	carry_look_ahead_4_bit cla4(a[15:12], b[15:12], c3, sum[15:12], cout);
endmodule

module carry_look_ahead_4_bit(a, b, cin, sum, cout);
	input[3:0] a, b;
	input cin;
	output[3:0] sum;
```

```
	output cout;
	wire [3:0] P, G, C;
	assign #10 P = a ^ b;
	assign #10 G = a & b;
	assign #10 C[0]= cin;
	assign #10 C[1]= G[0] | (P[0] & C[0]);
	assign #10 C[2]= G[1] | (P[1] & G[0]) | P[1] & P[0] & C[0];
	assign #10 C[3]= G[2] | (P[2] & G[1]) | P[2] & P[1] & G[0] | P[2] & P[1] & P[0] &
C[0];
	assign #10 cout= G[3] | (P[3] & G[2]) | P[3] & P[2] & G[1] | P[3] & P[2] & P[1] &
G[0] | P[3] & P[2] & P[1] & P[0] & C[0];
	assign #10 sum = P ^ C;
endmodule

module testbench_carry_look_ahead_16_bit;
	reg [15:0] a,b;
	reg cin;
	wire [15:0] sum;
	wire cout;
	carry_look_ahead_16_bit cla16bit(a, b,cin,sum,cout);
	initial
		begin
	a=0; b=0; cin=0;
	#100 a= 16'b0000000111110000; b=16'b000101100001100; cin=1'b0;
	#100 a= 16'b1100011010000101; b=16'b000000010000001; cin=1'b1;
	#100 a= 16'b0000001111110000; b=16'b011100000000000; cin=1'b0;
	#100 a= 16'b0000000000010000; b=16'b000010000000001; cin=1'b1;
	end
initial #500 $finish;
initial $dumpvars;
endmodule
```



A：前瞻式進位加法器 (CLA) 較 漣波進位加法器 (RCA)快。

C： RCA, 因為他所用用到的晶片面積複雜度比較低。

D：漣波加法器 ： 速行運算時速度慢，但運用的邏輯閘數比較少，因此布局簡單較容易設計。

前瞻加法器： 執行運算起起來速度快，但所需要用用到的邏輯閘數比較多。如果運算的位元數較多的話，電路會變得十分複雜。